

GPU-Accelerated DEM Simulation of a Snowball Avalanche Dynamics

on NVIDIA Maxwell

Maurizio Grisotto

Politecnico di Torino - GPU Programming 2025/2026

s336145@studenti.polito.it

Project: Angry Santa



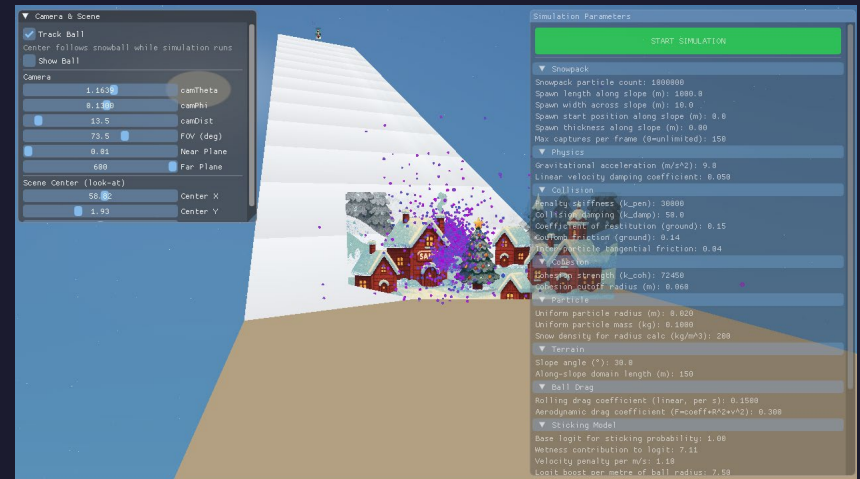
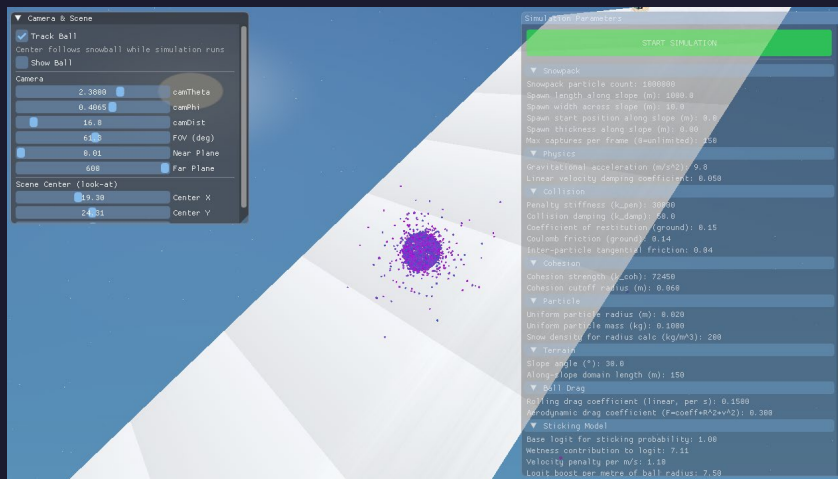
Outline

- Project Overview
- Physics Model
- System Architecture
- CPU Implementations
- GPU Optimizations
- Experimental Setup
- Results and Benchmarks
- Conclusion and Future Work

Project Overview

Introduction and Motivations

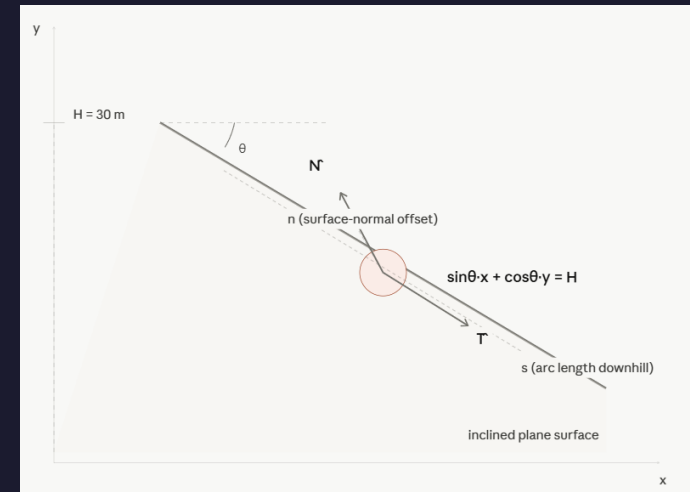
- Explore the concepts of the course.
- Simulation of a physical process - a snowball rolling down an inclined plane - as if it were a component of a real-time graphics engine for video games.
- Humorous scenario: Santa Claus rolls a giant snowball down a slope, destroying the elves' village.
- Rendering techniques using the Vulkan framework.
- Working code for Intel (Windows OS) and ARM (Linux OS) platforms.
- Experimenting with AI capabilities in a foreign domain



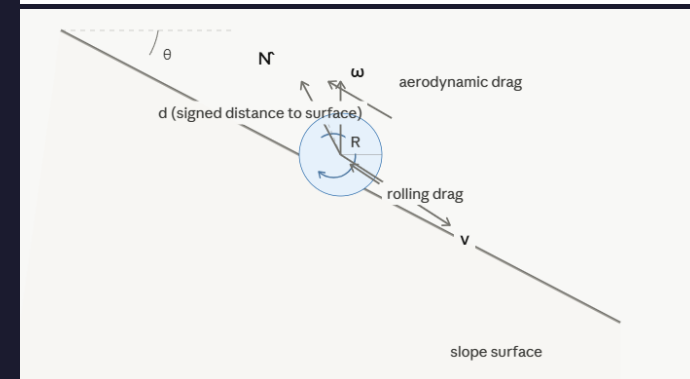
Physics Model

The (simplified) Discrete Element Method (DEM) with penalty-based contact and Coulomb friction without persistent contact history

- Snowpack: N particles on a rectangular band with jitter
Infinite inclined plane: $\sin(\theta) \cdot x + \cos(\theta) \cdot y = H$
Curvilinear parameterization (s, n) mapping
Per-particle wetness $w \in [w_min, w_max]$ via curand
Sorted once by posX $\rightarrow$ enables binary-search range limiting
- Dynamic shell:
Growing particle system where actual interactions happen
- Snowball core: rigid-body dynamics on slope
Rolling drag (linear) + aerodynamic drag (quadratic, $\propto R^2$)
Rolling-without-slip: $\omega = (1/R) \cdot (N_slope \times v)$
Quaternion integration for orientation



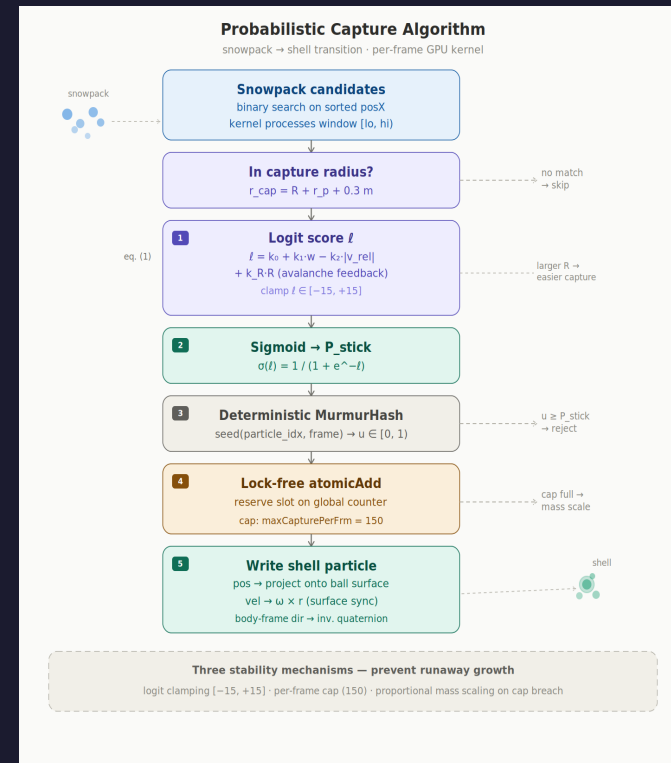
$$x = s \cos \theta + n \sin \theta, \quad y = \frac{H}{\cos \theta} - s \sin \theta + n \cos \theta$$



Physics Model

Probabilistic Capture Algorithm and Shell Forces

- Capture converts snowpack → shell particles each frame:
 - Logit: $\ell = k_0 + k_1 \cdot w - k_2 \cdot |v_{rel}| + k_R \cdot R$ (avalanche feedback)
 - Sigmoid probability: $P_{stick} = \sigma(\ell) = 1/(1+e^{-\ell})$
 - Deterministic murmurhash PRNG (no curand state overhead)
 - Lock-free atomicAdd for slot reservation
- Shell particle forces:
 - Gravity + Mass-proportional damping
 - Tether spring-damper to ball surface anchor
 - DEM inter-particle contact with Coulomb friction
- Stability: logit clamping, per-frame cap, proportional mass scaling



Physics Model

Default Values

Physics

```
gravity      = 9.81f;      // m/s2
dt           = 0.0033f;    // timestep (s)
damping      = 0.05f;      // mass-proportional velocity damping (k_d)
stiffness    = 30000.0f;   // soft-sphere penalty stiffness (k_pen)
collisionDamping = 50.0f;   // collision damping (k_damp)
cohesion     = 72450.0f;   // cohesion strength (k_coh)
cohesionRadius = 0.06f;    // cohesion cutoff (m)
particleRadius = 0.02f;    // uniform radius (m)
particleMass = 0.1f;       // per-particle mass (kg)
restitution  = 0.15f;      // ground bounce factor (e)
friction     = 0.14f;      // ground Coulomb friction ( $\mu_{\text{ground}}$ )
particleFriction = 0.04f;  // inter-particle friction ( $\mu_{\text{p}}$ )
snowDensity  = 200.0f;     // kg/m3
```

Slope

```
slopeAngleDeg = 30.0f;
slopeHeight   = 30.0f;    // plane offset (m)
domainLength  = 150.0f;   // max along-slope travel
```

Capture / Sticking

```
stickK0      = 1.0f;      // base logit  $\rightarrow$  ~73% at  $v=0$ ,  $w=1$ 
stickK1      = 7.11f;     // wetness boost in logit
stickK2      = 1.1f;      // velocity penalty (per m/s of  $|v_{\text{rel}}|$ )
stickRadiusBoost = 7.5f;   // avalanche feedback (logit boost per metre of ball radius)
logitClamp   = 15.0f;     // max absolute logit value for sigmoid stability
maxCapturePerFrm = 150;   // max particles captured per frame (growth rate limiter)
wetnessMin   = 0.0f;      // min initial wetness  $[0,1]$ 
wetnessMax   = 1.0f;      // max initial wetness  $[0,1]$ 
```

Physics Model

Default Values

Shell

```
shellTetherK      = 2717.0f;    // tether spring stiffness (N/m)
shellTetherDamp   = 50.0f;     // tether damping coefficient
```

Snowpack Geometry

```
numParticles      = 1000000;   // total snowpack particle count
spawnLengthS      = 1000.0f;   // patch along slope (m)
spawnWidthZ       = 10.0f;     // patch lateral width (m)
spawnStartS       = 0.0f;      // start offset along slope (m)
spawnThicknessN   = 0.0f;      // 0 = single layer on surface
```

Rigid-Body Drag

```
rollingDrag       = 0.15f;     // rolling friction (linear drag coefficient)
aeroCoeff          = 0.3f;      // aerodynamic drag coefficient (quadratic, scales with R2)
```

Spatial Grid

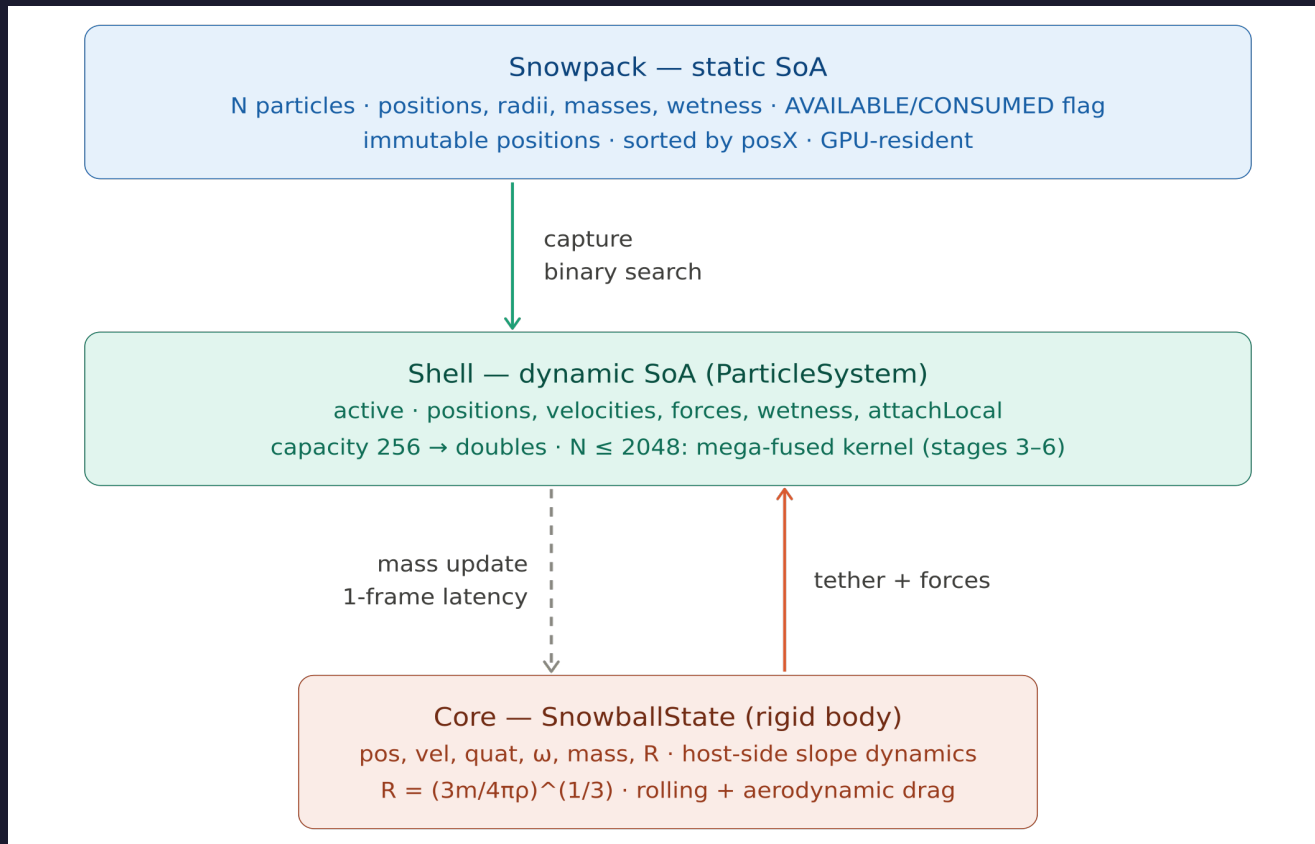
```
cellSize          = 4 * particleRadius; // = 0.08 m (default)
```

Performance / Profiling

```
traceInterval     = 0;         // frames between trace snapshots (0 to disable)
logInterval       = 0;         // frames between log outputs (0 = no log)
```

System Architecture

Three-Tier Particle Design



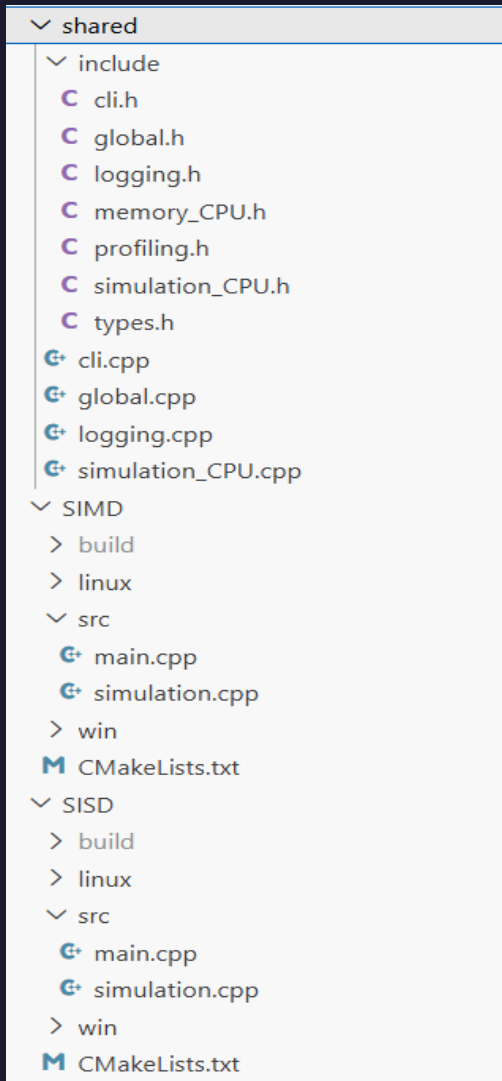
Deferred capture readback: 1-frame latency (pinned host memory)

CPU Implementations

Flynn's Taxonomy

- SISD Baseline:
 - Compiled with -O0 / -O1 - no inlining, no vectorization
 - Strict performance floor for speedup measurements
- SIMD + OpenMP optimized:
 - SSE2 (x86-64) / NEON (ARM64) - 4 particles per SIMD lane
 - 16-byte aligned SoA arrays (aligned_malloc)
 - OpenMP: #pragma omp parallel
 - $O(N^2)$ brute-force collision (no spatial hashing)

CPU Implementations



```
static int prevN = 0;
if (prevN > 0) {
    #ifdef HAS_OPENMP
        #pragma omp parallel for schedule(static)
    #endif
    for (int i = 0; i < prevN; i++) {
        uint32_t hash = grid.cellHash[i];
        if (i == 0 || hash != grid.cellHash[i - 1]) {
            grid.cellStart[hash] = INACTIVE;
            grid.cellEnd[hash] = INACTIVE;
        }
    }
}
```

```
#if CPU_SIMD_X86
// --- SSE path: 4 particles per iteration -----
const __m128 vSn = _mm_set1_ps(sn);
const __m128 vCs = _mm_set1_ps(cs);
const __m128 vS0 = _mm_set1_ps(S0);
const __m128 vPr = _mm_set1_ps(pr);
const __m128 vTwoPr = _mm_set1_ps(2.0f * pr);
const __m128 vHalfWz = _mm_set1_ps(-Wz * 0.5f);
const __m128 vY0 = _mm_set1_ps(H / cs);

const __m128 vMass = _mm_set1_ps(params.particleMass);
const __m128 vWmin = _mm_set1_ps(params.wetnessMin);
const __m128 vWdiff = _mm_set1_ps(params.wetnessMax - params.wetnessMin);

const int N4 = N & ~3;

#ifdef HAS_OPENMP...
#endif
for (int i = 0; i < N4; i += 4) { ...

    int scalarStart = N4;
#elif CPU_SIMD_NEON
// --- NEON path: 4 particles per iteration -----
const float32x4_t vSn = vdupq_n_f32(sn);
const float32x4_t vCs = vdupq_n_f32(cs);
const float32x4_t vY0 = vdupq_n_f32(H / cs);
const float32x4_t vPr = vdupq_n_f32(pr);
const float32x4_t vMass = vdupq_n_f32(params.particleMass);

const int N4 = N & ~3;
```

GPU Optimizations

Memory System

- SoA Layout: 100% coalesced 128-byte transactions per warp
AoS would waste up to 90% of bus bandwidth
- Constant Memory: SimParams (~152 B) broadcast to all warps in ~5 cycles
- `__ldg()` / `__restrict__`: routes loads through read-only texture cache
Doubles effective cache bandwidth without conflicts
- Pinned Memory: eliminates 30–40 ms GPU stalls from pageable memcpy
Capture count + mass packed into single 8-byte buffer
- Unity Build (coding): single translation unit shares `__constant__` across kernels

Memory Hierarchy

Every level of the hierarchy is explicitly exploited

Level	Usage	Size/Capacity
Registers	Local vars, force accumulators	24–48 / thread
__constant__	SimParams (broadcast)	~152 B
Shared Memory	Warp-cooperative tile	12.4 KB/block
Read-only Cache	__ldg() loads (immutable SoA)	HW-managed
L2 Cache	Particle working set	1 MB / 256 KB
Global (GDDR5)	SoA arrays	16N × 4 B

GPU Optimizations

Compute and ALU

- Warp-Cooperative Shared-Memory Tiling:
 - Each warp loads 32-neighbor tile into shared memory
 - [MAX_WARPS][WARP_SIZE+1] padding avoids bank conflicts
 - __syncwarp() instead of __syncthreads() (avoids deadlock)
- Non-symmetric collision: zero atomics, all forces in registers
 - Trades 2× FLOPs for zero contention (net win on Maxwell)
- Single precision + --use_fast_math (FP64:FP32 = 1:32 on Maxwell)
- ALU tricks: rsqrtf pattern, branchless clamping (FMNMX),
murmurhash PRNG, quaternion Rodrigues (15 FMA vs 27 matmul)

GPU Optimizations

Spatial Hashing - Multi-Stream

- Spatial Hashing and Uniform Grid: $O(N^2) \rightarrow O(N \cdot k)$ collision detection
Prime hash: $(cx \cdot 73856093) \oplus (cy \cdot 19349663) \oplus (cz \cdot 83492791)$
Compact table: $\text{nextPowerOf2}(N \times 8) \rightarrow \sim 256 \text{ KB}$ vs $\sim 75 \text{ MB}$ full domain
CUB DeviceRadixSort with bit-count limiting
Cell Boundaries: Index pointers store the start and end of each cell to quickly locate neighboring particles
Interaction Kernel: One thread per particle evaluates physical contact forces within neighboring cells
- Multi-Stream Pipeline:
simStream + gridStream + traceStream (async D2H)
- Gated Event Recording: only every 64 frames (saves $\sim 110\text{--}165 \mu\text{s}$)

GPU Pipeline

Adaptive Dual-Path Execution

- Small-N path ($N \leq 2048$): single mega-fused kernel
 - Replaces stages 3–6 in one GPU launch
 - Reduces GPU commands from ~ 10 to 1 per frame
 - Forces never transit global memory — stay in registers
- Large-N path ($N > 2048$): 7-stage multi-stream pipeline
 - Capture $\rightarrow$ Core update $\rightarrow$ Forces+Tether $\rightarrow$ Grid build $\rightarrow$ Collision $\rightarrow$ Integrate $\rightarrow$ Core RB
 - Forces+Tether || Grid build (concurrent on simStream / gridStream)
 - Fork/join via cudaEventDisableTiming events
- Threshold $N=2048$ chosen empirically:
 - Below: launch overhead $>$ brute-force $O(N^2)$ cost
 - Above: grid-based $O(N \cdot k)$ justifies build overhead

GPU Pipeline Large-N Stages

Multi-Stream Fork/Join Architecture

#	Stage	Kernel	Stream	Block
1	Particles Capture	captureFromSnowpack	sim	256
2	Core update	host code	—	—
3	Forces + Tether	forceTether	sim	256
4	Grid build	hash→sort→ranges	grid	256
5	Collision	neighborCollision	sim	512
6	Integrate + Ground	integrateGround	sim	256
7	Core Rigid Body	host code	—	—

GPU Pipeline Small-N Stages

Kernel Fusion Architecture

#	Stage	Kernel	Stream	Block
1	Particles Capture	captureFromSnowpack	sim	256
2	Core update	host code	—	—
3 – 4 – 5 - 6	Fusion	shellPhysicsFused	sim	256
7	Core Rigid Body	host code	—	—

Experimental Setup

Two Maxwell-class test platforms

Spec	Laptop (GTX 960M)	Jetson Nano
CPU	Intel i7-6700HQ	ARM Cortex-A57
CPU Cores/Threads	4 / 8	4 / 4
GPU	GeForce GTX 960M	Tegra X1
Compute Capability	SM 5.0	SM 5.3
SMs	5	1
CUDA Cores	640	128
L2 Cache	1 MB	256 KB
Memory BW	~30 GB/s (GDDR5)	~25.6 GB/s (LPDDR4)

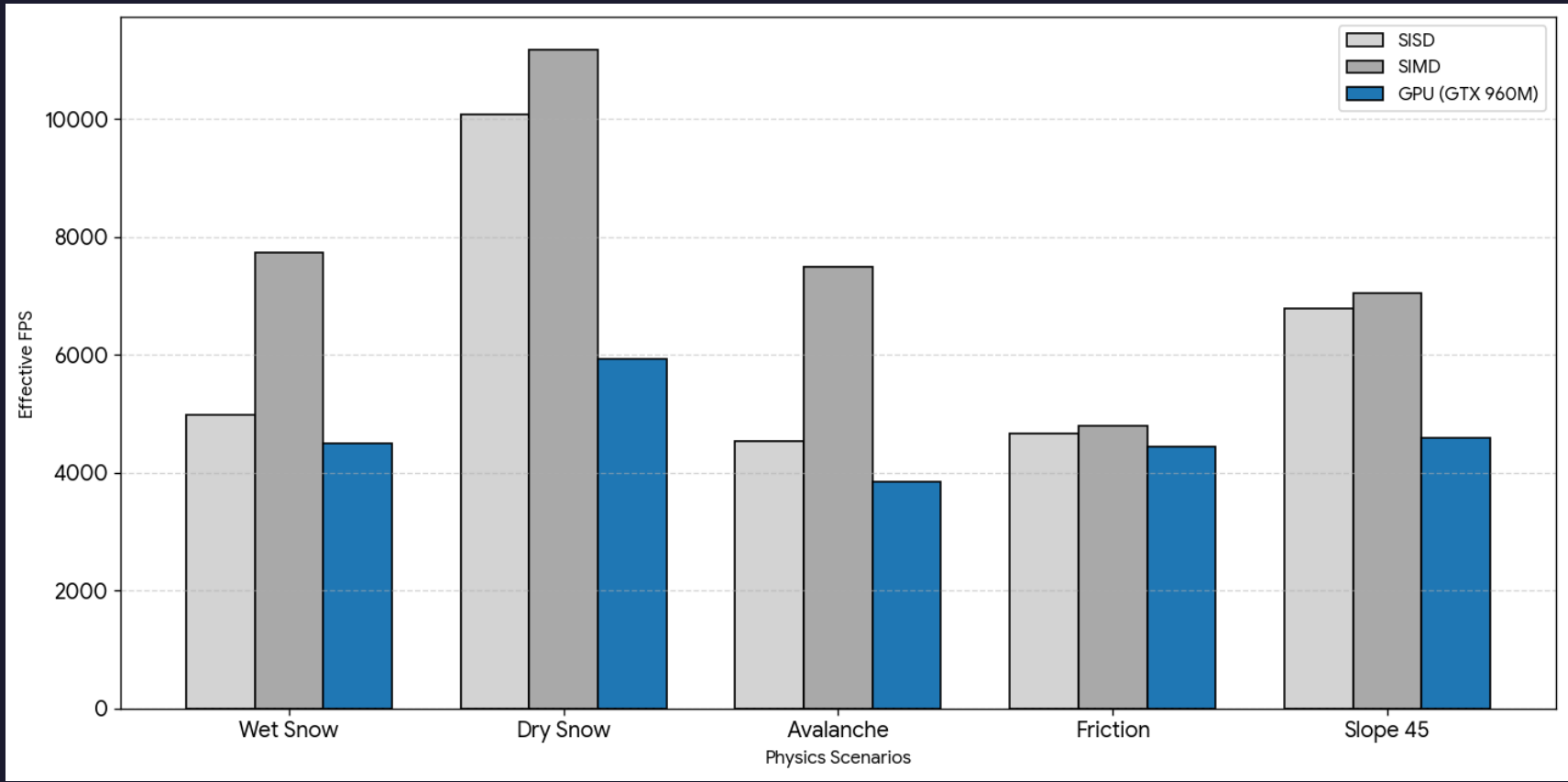
Results - GTX 960M

Per-Kernel Timing at $N = 100K$ (Fused Regime)

- At $N=10^5$, GPU consistently yields lower FPS than CPU baselines
- Shell never exceeds $\sim 2,048$ particles $\rightarrow$ single thread-block
 - Only 1 of 5 SMs utilized $\rightarrow$ 80% of GPU idle
 - Fixed costs (launch, sync, PCIe) = 35–50% of frame time
- CPU advantage: deep OoO pipeline, large L3 cache keeps all data in cache
- SIMD: 4-wide AVX vectorization $\rightarrow$ peak 11,174 FPS
- Key insight: GPU parallelism only pays off once workload amortizes launch and transfer overheads and if is sufficient «big»
- Crossover: between $N=100K$ and $N=500K$ on this platform

Results - GTX 960M Scenarios N=10⁵

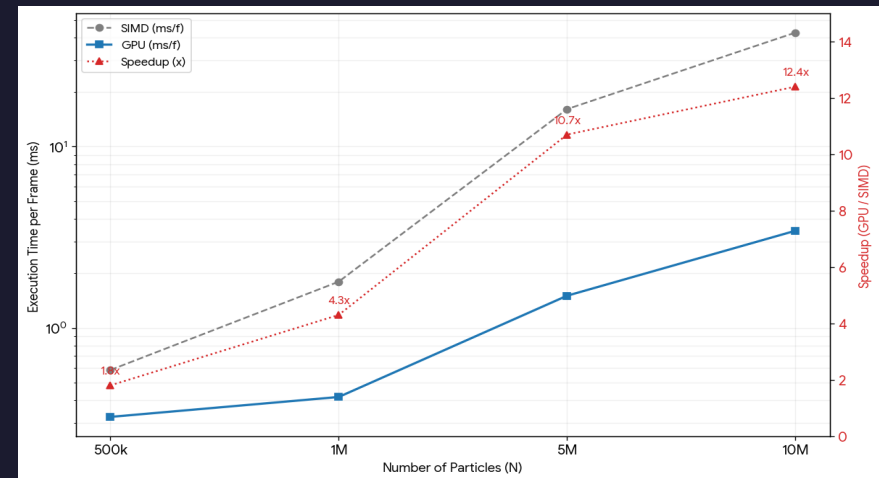
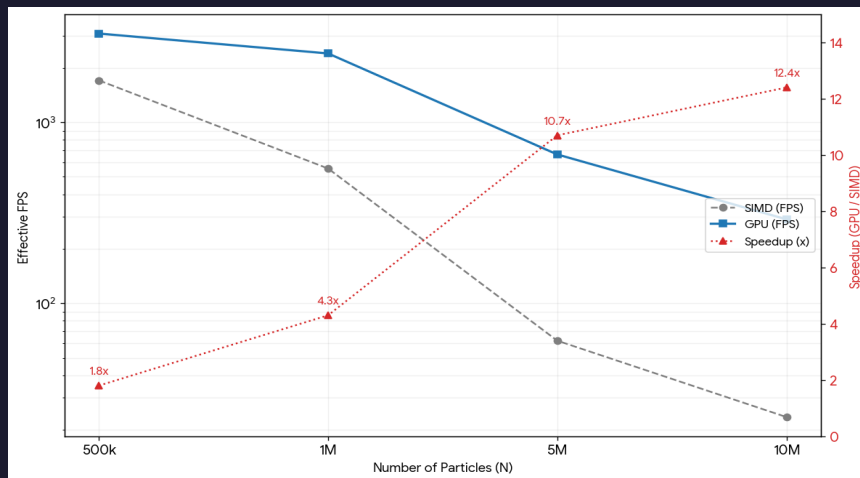
GPU parallelism only pays off once workload amortizes launch and transfer overheads and if is sufficient “big”



Results - GTX 960M Scaling

GPU vs SIMD CPU - Speedup grows with particle count

N	SIMD (FPS)	GPU (FPS)	SIMD (ms/f)	GPU (ms/f)	Speedup
500K	1,706.6	3,097.8	0.586	0.323	1.8×
1M	555.8	2,404.5	1.799	0.416	4.3×
5M	62.1	664.7	16.10	1.505	10.7×
10M	23.5	291.1	42.55	3.435	12.4×



→ 12.4× speedup at N=10M: GPU's $O(N \cdot k)$ grid-based collision scales far better than CPU's $O(N^2)$
→ Overhead never exceeds 3% of frame time at large scale

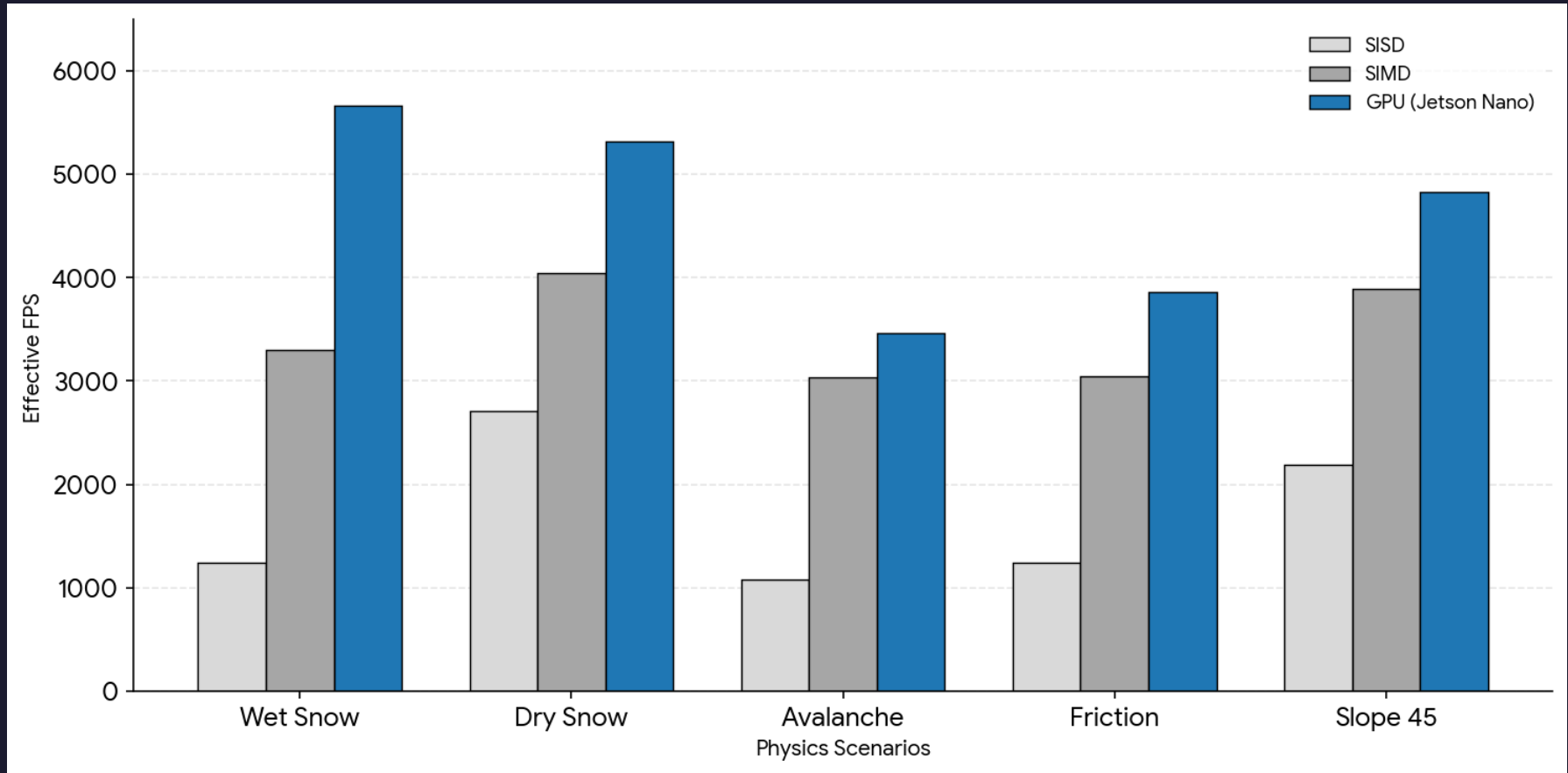
Results - Jetson Nano

Unified Memory: GPU Wins at Small N

- At $N=10^5$: GPU slightly outperforms CPU in all scenarios (vs GTX result)
 - Unified memory eliminates PCIe transfer overhead entirely
 - Kernel launch overhead: only 11–18% of frame time (vs 35–50% on GTX)
- Scaling bottleneck at large N:
 - Single SM (128 cores, max 2048 threads) → serializes collision
 - $N=5M$: collision kernel 174.9 ms (GPU) vs 37.0 ms (4-core SIMD CPU)
 - GPU speedup inverts: 0.3× at $N=5M$

Results - Jetson Nano Scenarios N=10⁵

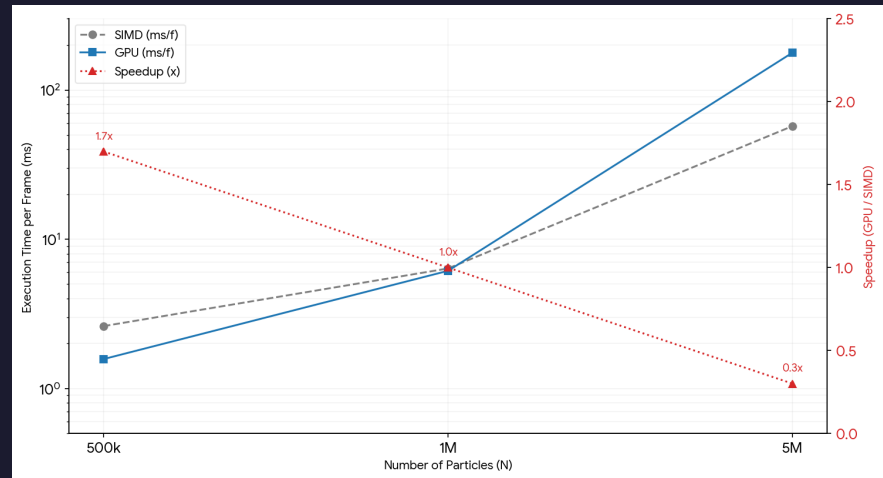
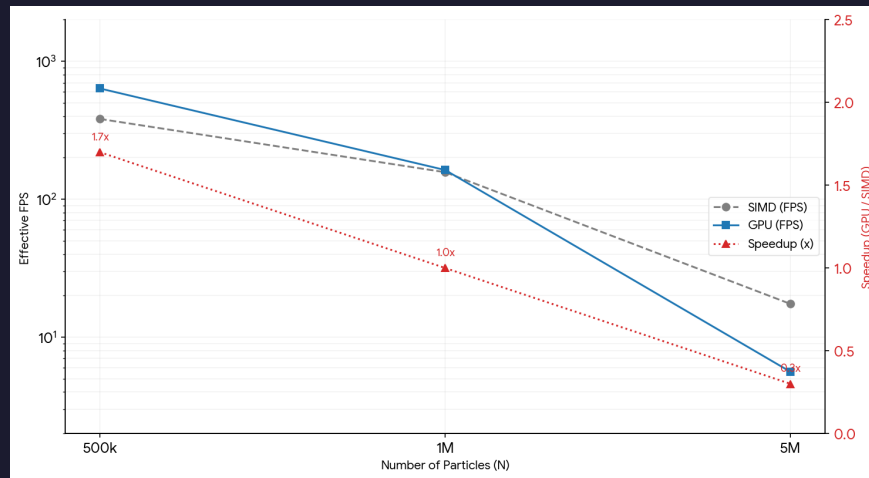
Unified Memory: GPU Wins at Small N



Results - Jetson Nano Scaling

Single SM bottleneck at large N

N	SIMD (FPS)	GPU (FPS)	SIMD (ms/f)	GPU (ms/f)	Speedup
500K	382.0	635.4	2.618	1.574	1.7×
1M	156.9	162.8	6.372	6.144	1.0×
5M	17.4	5.6	57.31	177.95	0.3×
10M	—	—	—	—	OOM



Key insight: multi-SM GPUs (GTX 960M with 5 SMs) mitigate single-SM saturation at large N

Final Key Findings

Performance Analysis Summary

- GTX 960M (5 SMs):
 - CPU wins at $N \leq 100K$ (launch overhead dominates)
 - GPU crossover at $N \sim 500K \rightarrow$ reaches 12.4× at $N=10M$
 - Grid-based $O(N \cdot k)$ collision is the key scalability enabler
- Jetson Nano (1 SM):
 - GPU wins at small N (no PCIe overhead)
 - Single SM saturates at $N \geq 1M \rightarrow$ CPU catches up and overtakes
- Both platforms:
 - Overhead (launch, sync, transfers) < 3% at scale
 - Mega-fused kernel essential for small- N performance
 - Architecture-aware tuning >> generic optimization

Conclusion

Key Takeaways

- Architecture-aware optimization matters
 - Every register, KB of shared memory, and kernel launch is measurable
 - Warp-cooperative tiling, zero-spill allocation, gated events
 - all responses to specific Maxwell bottlenecks
- Adaptive fusion pays off
 - Mega-fused ($N \leq 2048$) → seamless transition to grid-based ($N > 2048$)
 - Eliminates launch overhead during initial growth phase
- Zero spill > 100% occupancy
 - 62.5–75% occupancy acceptable to avoid DRAM spill (~300 cycles)
 - Better real-world performance than forcing higher occupancy

Future Work

Extensions and Improvements

- Multi-GPU / Domain Decomposition
 - Partition spatial grid across GPUs via halo exchange
 - Theoretical analysis discussed in the paper attachment
- Persistent-Contact DEM
 - Track tangential spring history across frames
 - More accurate friction modelling (additional per-pair state)
- 3D Terrain
 - Height-map or mesh-based terrain for realistic mountain geometry
- Volta+ Optimizations (SM 7.0+)
 - Hardware atomicAdd(float) → symmetric collision viable again
 - Cooperative groups replace warp-level synchronization

Thank You

Questions?



Code: github.com/politos336145/GPU-Programming-2025-2026

Contact: s336145@studenti.polito.it